

AUTOM-97 LAB: Monaco

Configuration as Code

Series: AUTOM — Dynatrace Automation | **Reference:** 97 — Monaco Hands-On LAB | **Created:** April 2026 | **Last Updated:** 10/02/2026

Overview

Hands-on lab for installing Monaco CLI, downloading existing Dynatrace configuration, modifying it, validating, and deploying to a target environment. Includes CI/CD pipeline integration with GitHub Actions. Complete the lecture notebook **AUTOM-03: Monaco Configuration-as-Code** first, then work through these exercises step by step.

Table of Contents

1. [Install Monaco CLI](#)
 2. [Create Access Token](#)
 3. [Create Manifest File](#)
 4. [Download Existing Configuration](#)
 5. [Explore Downloaded Configuration](#)
 6. [Create a New Configuration](#)
 7. [Validate Configuration](#)
 8. [Deploy Configuration](#)
 9. [Delete Configuration](#)
 10. [Multi-Environment Promotion](#)
 11. [GitHub Actions CI/CD Pipeline](#)
 12. [Summary & Checklist](#)
-

Prerequisites

Requirement	Details
Completed	AUTOM-03: Monaco Configuration-as-Code (lecture notebook)
Dynatrace Environment	SaaS tenant (Grail enabled)
Permissions	Account with ability to create API tokens
Local Tools	Terminal with <code>curl</code> and <code>chmod</code> (Monaco has no Homebrew formula)
Git	Git CLI installed (for CI/CD section)
GitHub Account	Required for Section 11 (CI/CD pipeline)

1. Install Monaco CLI

Monaco is a standalone binary with no runtime dependencies. Choose the release asset for your platform — Monaco is distributed only as GitHub release binaries (there is no Homebrew formula).

macOS (Apple silicon)

```
# Download the latest Monaco binary for macOS (Apple silicon)
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-darwin-arm64 -o monaco
chmod +x monaco
sudo mv monaco /usr/local/bin/
# Intel Macs: use monaco-darwin-amd64
```

Linux (x86-64)

```
# Download the latest Monaco binary for Linux (amd64)
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco

# Make it executable and move to PATH
chmod +x monaco
sudo mv monaco /usr/local/bin/

# Other Linux architectures: monaco-linux-arm64, monaco-linux-386
```

Windows (PowerShell)

```
# Download Monaco for Windows
Invoke-WebRequest -Uri "https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-windows-amd64.exe" -OutFile "monaco.exe"

# Move to a directory in your PATH
Move-Item monaco.exe C:\Tools\monaco.exe
```

Verify Installation

```
# Verify Monaco is installed and check version
monaco version
```

Note: Monaco releases are available at github.com/Dynatrace/dynatrace-configuration-as-code/releases. Check for the latest version before installing.

2. Create Access Token

This LAB works on a **service-level objective** (type: slo-v2), a platform configuration type, so it needs a **platform token** as well as the classic access token Monaco uses for Settings 2.0.

Platform Token Scopes (required for this LAB)

Scope	Purpose
<code>slo:slos:read</code>	Download SLOs (<code>--only-slo-v2</code>)
<code>slo:slos:write</code>	Deploy and delete SLOs

Add the scopes for the other platform types only if the project will manage them — workflows, documents, segments and buckets each have their own `<service>:<object>:read / :write` pair. Create the token under **Account Management** → **Identity & access management** → **Platform tokens**, ideally on a service user (AUTOM-04 §3).

The SLO scope names are the ones the Terraform provider's `platform_slo` resource docs list ("View SLOs (`slo:slos:read`) and Create and edit SLOs (`slo:slos:write`)"). Checking the deployed SLO's evaluation in the app is done as *you*, so your own user needs read access to the data its SLI queries.

Access Token Scopes (Settings 2.0 and classic configuration)

Scope	Purpose
<code>Read settings (settings.read)</code>	Download Settings 2.0 objects
<code>Write settings (settings.write)</code>	Deploy Settings 2.0 objects
<code>Read SLO (slo.read)</code>	<i>Classic</i> SLOs only — not needed for <code>slo-v2</code>
<code>Write SLO (slo.write)</code>	<i>Classic</i> SLOs only — not needed for <code>slo-v2</code>
<code>Read configuration (ReadConfig)</code>	Download classic API configs
<code>Write configuration (WriteConfig)</code>	Deploy classic API configs
<code>Access problem and event feed, metrics, and topology (DataExport)</code>	Entity and metric access

3. Create Manifest File

The `manifest.yaml` is the entry point for every Monaco project. It defines which projects to manage and which environments to target.

Project Directory Setup

```
# Create the project directory structure
mkdir -p dynatrace-config/my-project
cd dynatrace-config
```

Manifest Format

Create `dynatrace-config/manifest.yaml` with this content:

```
manifestVersion: "1.0"

projects:
  - name: my-project
    path: my-project

environmentGroups:
  - name: default
    environments:
      - name: dev
        url:
          type: environment
          value: DT_ENV_URL
        auth:
          token:
            name: DT_API_TOKEN
          platformToken:
            name: DT_PLATFORM_TOKEN
```

Field Reference

Field	Description
<code>manifestVersion</code>	Always <code>"1.0"</code> for current Monaco
<code>projects[].name</code>	Logical name for the project

Field	Description
<code>projects[].path</code>	Relative path to the project directory
<code>environmentGroups[].name</code>	Group name (used for overrides)
<code>environments[].name</code>	Environment identifier (used with <code>--environment</code> flag)
<code>environments[].url.value</code>	Environment variable name containing the tenant URL
<code>environments[].auth.token.name</code>	Environment variable name containing the API (access) token — settings and classic configuration APIs
<code>environments[].auth.platformToken.name</code>	Environment variable name containing a platform token (Monaco 2.24.0+) — needed for workflows, documents, Grail buckets and segments. An <code>oAuth</code> client (<code>clientId.name</code> / <code>clientSecret.name</code>) is the alternative; you cannot define both

Important: The `url.value` and `auth.token.name` fields reference **environment variable names**, not the actual URL or token values. Monaco reads the values from your shell environment at runtime. Access tokens and platform tokens are not interchangeable — with only `token` defined, `monaco download` returns Settings 2.0 and classic configurations but none of the platform types listed in §4 — including the SLOs this LAB works on.

4. Download Existing Configuration

Downloading is the fastest way to get started. Monaco exports your current tenant configuration into versioned YAML and JSON files.

Download All Configuration

```
# Download all configuration from the dev environment
cd dynatrace-config
monaco download manifest.yaml --environment dev --output-folder downloaded
```

Download Specific Configuration Types

```
# Download only SLOs and segments (platform types – needs auth.platformToken)
cd dynatrace-config
monaco download manifest.yaml --environment dev --output-folder downloaded \
  --only-slo-v2 --only-segments

# Classic inventory, while the tenant still has them: specific Settings 2.0
schemas
# monaco download manifest.yaml --environment dev --output-folder downloaded \
#   --settings-schema builtin:alerting.profile,builtin:management-zones
```

What Gets Downloaded

Monaco organizes downloaded configuration by type:

Config type	Description	Examples
slo-v2	SLOs on the modern SLO app	SLO Service API objects
segment	Filter segments	Segment definitions
automation	Workflows, business calendars, scheduling rules	Automation API objects
document	Dashboards, notebooks, launchpads	Document API objects
bucket	Grail buckets	Bucket definitions

Config type	Description	Examples
settings	Settings 2.0 objects	builtin:davis.anomaly-detectors , builtin:ownership.teams ; classic estates also builtin:alerting.profile , builtin:management-zones
classic	Classic Config API v1	Request attributes, dashboards (classic)

The type names are Monaco's own config-type identifiers (`pkg/config/types.go`). The matching `--only-*` download flags are `--only-slo-v2` , `--only-segments` , `--only-automation` , `--only-documents` , `--only-buckets` , `--only-settings` , `--only-apis` and `--only-openpipeline` , and can be combined. `--only-openpipeline` is marked **Deprecated** in the [Monaco command reference \(DT docs\)](#): OpenPipeline configuration is Settings 2.0 objects, so download it with `--only-settings` or `--settings-schema` with each `builtin:openpipeline.*` schema in use, plus `--admin-access` for pipelines other users own.

Platform types need platform auth. `slo-v2` , `document` , `automation` , `bucket` and `segment` configs come through the Platform APIs, so they appear only when the manifest environment has `auth.platformToken` or `auth.oAuth` (see §3).

Note: Some classic API types (AWS/Azure/K8s credentials, Extensions v1) support deploy but **not** download. See the [Monaco API coverage](#) for details.

5. Explore Downloaded Configuration

After downloading, examine the directory structure and understand the file format.

Directory Structure

```
# View the downloaded directory structure
cd dynatrace-config
find downloaded -type f | head -30
```

Expected output looks like:

```
downloaded/
  my-project/
    slo-v2/
      config.yaml
      <slo-name>.json
    segment/
      config.yaml
      <segment-name>.json
    ...
```

Folder names follow the config type — a Settings 2.0 download (the classic inventory command above) produces one folder per schema instead, such as `builtin:management-zones/`. Check your own download rather than relying on the exact names here.

Example config.yaml

Each config type directory contains a `config.yaml` that references JSON template files:

```
configs:
  - id: checkout-availability
    config:
      name: "Checkout availability"
      template: checkout-availability.json
      type: slo-v2
```

Key Fields

Field	Description
<code>id</code>	Unique identifier for this config within the project
<code>config.name</code>	Display name in the Dynatrace UI
<code>config.template</code>	Path to the JSON payload file
<code>type</code>	The config type — <code>slo-v2</code> , <code>segment</code> , <code>{automation: {resource: workflow}}</code> , <code>{document: {kind: dashboard}}</code> , ...
<code>type.settings.schema</code> / <code>type.settings.scope</code>	For Settings 2.0 configs only: the schema ID and the deployment scope (<code>environment</code> , <code>host</code> , <code>service</code> , ...)

Examine a JSON Template

```
# View an example JSON template (replace path with an actual downloaded file)
cd dynatrace-config
cat downloaded/my-project/slo-v2/<slo-name>.json | python3 -m json.tool
| head -30
```

Tip: The JSON templates contain the exact payload that Monaco sends to the Dynatrace API. You can modify these files to change configuration values.

6. Create a New Configuration

Create a new **SLO** from scratch to understand how Monaco configurations are authored. An SLO is a good first target: it is a single self-contained payload, it changes nothing about how your tenant alerts, and you can see it evaluate in the SLO app minutes after deploying.

Classic equivalent — earlier versions of this LAB created an alerting profile here. That still works on a tenant that has not been upgraded, but `builtin:alerting.profile` is on Dynatrace's list of [Settings 2.0 schemas that are removed in Latest Dynatrace \(DT docs\)](#); its successor is a problem-triggered workflow (`type: {automation: {resource: workflow}}`). The classic walkthrough is kept as a labelled block at the end of this section.

Step 1: Create the Directory

```
# Create directory for SLO configs
mkdir -p dynatrace-config/my-project/slo-v2
```

Step 2: Create config.yaml

Create `dynatrace-config/my-project/slo-v2/config.yaml` :

```

configs:
  - id: checkout-availability
    config:
      name: "Checkout availability"
      template: checkout-availability.json
      parameters:
        slo_target: 99.0
      type: slo-v2

```

Step 3: Create the JSON Template

Create `dynatrace-config/my-project/slo-v2/checkout-availability.json` :

```

{
  "name": "{{ .name }}",
  "description": "Request success ratio, rolling 7 days – Monaco LAB",
  "tags": ["lab:monaco"],
  "criteria": [
    {
      "target": {{ .slo_target }},
      "warning": 99.5,
      "timeframeFrom": "now-7d",
      "timeframeTo": "now"
    }
  ],
  "customSli": {
    "indicator": "timeseries { total = sum(dt.service.request.count), failures = sum(dt.service.request.failure_count) }\n| fieldsAdd sli = ((total[] - failures[]) / total[]) * 100\n| fieldsRemove total, failures"
  }
}

```

Three things to notice: `criteria` is a **list** (an SLO can carry more than one window); the SLI is a **DQL query** that must produce a timeseries field named `sli` — this one is SLO-02's availability query without `from: / interval:`, which the criteria supply; and `{{ .slo_target }}` is left **unquoted** so the rendered JSON carries a number. The shape matches Monaco's own `slo-v2` integration-test template.

--dry-run will not catch a bad SLI. The template is valid JSON either way; a DQL error only surfaces when the SLO evaluates. Run the `indicator` query in a notebook first (add `, from:-7d` to the `timeseries` line there).

How References Work

Configurations reference each other with a **reference parameter** — `[project, configType, configId, property]`, or the long form below. For platform types `configType` is the type (`segment` , `slo-v2` , ...); for Settings 2.0 it is the schema ID. To scope the SLO to a segment deployed from the same project:

```
configs:
- id: checkout-availability
  config:
    name: "Checkout availability"
    template: checkout-availability.json
    parameters:
      slo_target: 99.0
      checkout_segment_id:
        type: reference
        configType: segment
        configId: checkout-production
        property: id
      type: slo-v2
```

...and add `"filterSegments": [{"id": "{{ .checkout_segment_id }}}"]` inside `customSli`. Monaco resolves the reference per environment at deploy time and deploys the segment first. (The `{{ .<type>.<id>.id }}` form is Monaco v1 syntax and does not load in v2.)

Classic — the alerting-profile version of this exercise

Dynatrace Classic — unupgraded tenants only. Kept for estates that still maintain alerting profiles. *"Alerting profiles and problem notifications are Dynatrace Classic."* Delay (`delayInMinutes`) has no field on the successor object; it becomes the workflow trigger's **Minimum duration** option.

Sources: [Upgrade from Classic problem notification to simple workflows \(DT docs\)](#) — *"The delay, update, and severity capabilities described in this guide exist only on the workflow trigger."*, [Problem and event triggers \(DT docs\)](#) — *"The Minimum duration option postpones the trigger until the problem has been open for at least the configured duration."*

`dynatrace-config/my-project/alerting-profiles/config.yaml` :

```

configs:
  - id: production-critical-alerts
    config:
      name: "Production Critical Alerts"
      template: production-critical-alerts.json
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment

```

Classic JSON template

Create `dynatrace-config/my-project/alerting-profiles/production-critical-alerts.json` :

```

{
  "name": "{{ .name }}",
  "severityRules": [
    {
      "severityLevel": "AVAILABILITY",
      "delayInMinutes": 0,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    },
    {
      "severityLevel": "ERRORS",
      "delayInMinutes": 5,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    },
    {
      "severityLevel": "PERFORMANCE",
      "delayInMinutes": 15,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    },
    {
      "severityLevel": "RESOURCE_CONTENTION",
      "delayInMinutes": 30,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    }
  ]
}

```

Classic reference example

Configurations reference each other with a **reference parameter** — a list of `[project, configType, configId, property]`. For Settings 2.0 configs, `configType` is the schema ID:

```
configs:
  - id: notification-with-profile
    config:
      name: "Slack Notification"
      template: slack-notification.json
      parameters:
        alerting_profile_id: ["my-project", "builtin:alerting.profile",
"production-critical-alerts", "id"]
      type:
        settings:
          schema: builtin:problem.notifications
          scope: environment
```

Monaco resolves these references during deployment, ensuring the correct runtime IDs are used; the template reads the value as `{{ .alerting_profile_id }}`. (The `{{ .<type>.<id>.id }}` form is Monaco v1 syntax and does not load in v2.)

7. Validate Configuration

Always dry-run before deploying. `--dry-run` parses the manifest and every config, checks that templates are valid JSON, and resolves references — **without contacting the tenant**, so it cannot catch payload, schema or permission errors.

Run Dry-Run Validation

```
# Check file structure and references without deploying (no API calls)
cd dynatrace-config
monaco deploy --dry-run manifest.yaml
```

What Dry-Run Checks

Check	Description
YAML syntax	Valid YAML in <code>manifest.yaml</code> and <code>config.yaml</code> files
JSON syntax	Valid JSON in template files
Reference resolution	Reference parameters (<code>[project, configType, configId, property]</code>) point at configs that exist

What it does **not** check: the template against the Settings 2.0 schema, connectivity, or token scopes. The Monaco commands reference is explicit — *"A dry-run doesn't connect to Dynatrace and can't validate the content of the JSON sent to Dynatrace."* A deploy can still fail with HTTP 400 after a clean dry-run.

Common Errors

Error	Surfaces at	Cause	Fix
template not found	dry-run	Wrong path in <code>config.template</code>	Use path relative to <code>config.yaml</code>
unresolved reference	dry-run	Referenced config ID does not exist	Check spelling and project path
invalid JSON	dry-run	Syntax error in template	Run <code>python3 -m json.tool <file></code> to find the error
unknown schema	deploy	Typo in schema ID, or a schema removed in Latest Dynatrace	Verify schema at Settings > Schema in Dynatrace
insufficient permissions	deploy	Token missing scopes	Add required scopes to the API token

8. Deploy Configuration

After validation passes, deploy to your target environment.

Deploy to All Environments

```
# Deploy to all environments defined in manifest.yaml
cd dynatrace-config
monaco deploy manifest.yaml
```

Deploy to a Specific Environment

```
# Deploy only to the dev environment
cd dynatrace-config
monaco deploy manifest.yaml --environment dev
```

Expected Output

Illustrative — exact log lines vary by Monaco version:

```
2026-04-04T10:00:00Z INFO Loading manifest "manifest.yaml"
2026-04-04T10:00:00Z INFO Projects to deploy: my-project
2026-04-04T10:00:01Z INFO Deploying config checkout-availability (slo-v2) to
dev...
2026-04-04T10:00:02Z INFO Deployed 1/1 configs successfully
```

Verify in Dynatrace

After deployment:

1. Open the **Service-Level Objectives** app
2. Confirm "Checkout availability" appears with the `lab:monaco` tag
3. Check that it evaluates — a status and error budget, not an error — and that the target reads 99 (the `slo_target` parameter)

Deploy a Specific Project

```
# Deploy only a specific project (useful when manifest has multiple projects)
cd dynatrace-config
monaco deploy manifest.yaml --project my-project
```

Tip: Monaco uses the config `id` as an idempotency key. Re-deploying the same config updates it in place rather than creating duplicates.

9. Delete Configuration

Monaco can remove configurations that are no longer needed.

Create a delete.yaml File

Create `dynatrace-config/delete.yaml`:

```
delete:
  - project: my-project
    type: slo-v2
    id: checkout-availability
```

Execute Delete

The `type` is Monaco's config type (`slo-v2`) — or, for a Settings 2.0 object such as the classic alerting profile, the schema ID (`builtin:alerting.profile`). Non-classic entries need `project` and `id` together.

```
# Delete configurations listed in delete.yaml
# Note: the manifest is passed via --manifest, not as a positional argument
cd dynatrace-config
monaco delete --manifest manifest.yaml --file delete.yaml --environment dev
```

Delete Behavior

Behavior	Description
Scoped to Monaco	Only deletes configs that Monaco deployed (matched by <code>id</code>)
Non-destructive	Configs created manually in the UI are never touched
Reversible	Re-deploy the config to restore it
Environment-specific	Use <code>--environment</code> to target a single environment

Warning: The `delete` command is irreversible for the target environment. Always use `--dry-run` first if you want to preview what would be removed.

10. Multi-Environment Promotion

A core Monaco strength is deploying the same configuration across dev, staging, and production with environment-specific overrides.

Multi-Environment Manifest

```
manifestVersion: "1.0"

projects:
  - name: my-project
    path: my-project

environmentGroups:
  - name: default
    environments:
      - name: dev
        url:
          type: environment
          value: DT_DEV_URL
        auth:
          token:
            name: DT_DEV_TOKEN
          platformToken:
            name: DT_DEV_PLATFORM_TOKEN
      - name: staging
        url:
          type: environment
          value: DT_STAGING_URL
        auth:
          token:
            name: DT_STAGING_TOKEN
          platformToken:
            name: DT_STAGING_PLATFORM_TOKEN
      - name: prod
        url:
          type: environment
          value: DT_PROD_URL
        auth:
          token:
            name: DT_PROD_TOKEN
          platformToken:
            name: DT_PROD_PLATFORM_TOKEN
```

Environment-Specific Overrides

Use `environmentOverrides` (per environment) or `groupOverrides` (per environment group) in `config.yaml` to customize values. Both sit at the config level, as siblings of `config:`

```
configs:
  - id: checkout-availability
    config:
      name: "Checkout availability"
      template: checkout-availability.json
      parameters:
        slo_target: 99.0
    type: slo-v2
    environmentOverrides:
      - environment: staging
        override:
          parameters:
            slo_target: 99.5
      - environment: prod
        override:
          parameters:
            slo_target: 99.9
```

Promotion Workflow

```
# Step 1: Deploy to dev and validate
cd dynatrace-config
monaco deploy manifest.yaml --environment dev

# Step 2: Deploy to staging after dev validation
monaco deploy manifest.yaml --environment staging

# Step 3: Deploy to production after staging validation
monaco deploy manifest.yaml --environment prod
```

Skip Configs in Certain Environments

`skip` is a boolean on `config:`. Default it to `false` and override it to `true` for the environments to exclude:

```

configs:
  - id: debug-dashboard
    config:
      name: "Debug Dashboard"
      template: debug-dashboard.json
      skip: false
    type:
      document:
        kind: dashboard
    environmentOverrides:
      - environment: prod
        override:
          skip: true

```

This deploys the debug dashboard to dev and staging but skips production entirely.

Validate Dashboard Configs Before Deploying Them

`monaco deploy --dry-run` is the pre-deploy gate. Run it before every real deploy; it resolves the manifest and references and checks template JSON without contacting the tenant, so a broken reference or an unresolved `type: environment` parameter with no `default` surfaces before it reaches a tenant.

For dashboard configs specifically, `--dry-run` is necessary but not sufficient: it validates the Monaco configuration around the template, not the dashboard document inside `template: <file>.json`. To Monaco that file is an opaque payload.

Forthcoming/rolling out (SaaS 1.344): a dashboard that fails validation will no longer load. Published 07/27/2026, staged tenant rollout from 07/29/2026 — verify whether it has reached your tenant. Before 1.344 a failing dashboard still loaded and surfaced validation warnings; after, it does not load at all, and Dynatrace names **API- and AI-authored** dashboards as the most affected population — which is what a Monaco-templated dashboard is. So after a dry-run passes, deploy to a non-production environment and **open the dashboard in the Dashboards app to confirm zero validation warnings** before promoting to prod. Prefer exporting a working UI-authored dashboard as your template over hand-writing the tile map. Full gate: **DASH-07 \$5**.

11. GitHub Actions CI/CD Pipeline

Automate Monaco deployments with a GitHub Actions workflow that validates on pull requests and deploys on merge to main.

Workflow File

Create `.github/workflows/monaco-deploy.yml` :

```
name: Deploy Dynatrace Config

on:
  push:
    branches: [main]
    paths: ['dynatrace-config/**']
  pull_request:
    branches: [main]
    paths: ['dynatrace-config/**']

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Validate (dry-run)
        run: monaco deploy --dry-run dynatrace-config/manifest.yaml
        env:
          DT_ENV_URL: ${ secrets.DT_ENV_URL }
          DT_API_TOKEN: ${ secrets.DT_API_TOKEN }
          DT_PLATFORM_TOKEN: ${ secrets.DT_PLATFORM_TOKEN }

  deploy:
    needs: validate
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Deploy to Dynatrace
        run: monaco deploy dynatrace-config/manifest.yaml
        env:
```

```
DT_ENV_URL: ${ secrets.DT_ENV_URL }
DT_API_TOKEN: ${ secrets.DT_API_TOKEN }
DT_PLATFORM_TOKEN: ${ secrets.DT_PLATFORM_TOKEN }
```

Workflow Explanation

Step	Trigger	What Happens
validate	Every PR and push to <code>main</code>	Installs Monaco, runs <code>--dry-run</code> to catch errors
deploy	Only on push to <code>main</code>	Installs Monaco, deploys all configurations

Configure GitHub Secrets

Add these secrets to your GitHub repository (**Settings > Secrets and variables > Actions**):

Secret Name	Value
<code>DT_ENV_URL</code>	Your Dynatrace tenant URL (e.g., <code>https://abc12345.live.dynatrace.com</code>)
<code>DT_API_TOKEN</code>	API token with Configuration as Code scopes
<code>DT_PLATFORM_TOKEN</code>	Platform token referenced by <code>auth.platformToken</code> in the manifest

Multi-Environment CI/CD

For promoting across environments, extend the workflow with separate jobs:

```

deploy-staging:
  needs: deploy
  runs-on: ubuntu-latest
  environment: staging
  steps:
    - uses: actions/checkout@v4
    - name: Install Monaco
      run: |
        curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
        chmod +x monaco
        sudo mv monaco /usr/local/bin/
    - name: Deploy to Staging
      run: monaco deploy dynatrace-config/manifest.yaml --environment staging
  env:
    DT_STAGING_URL: ${ secrets.DT_STAGING_URL }
    DT_STAGING_TOKEN: ${ secrets.DT_STAGING_TOKEN }

```

Use GitHub [Environments](#) with required reviewers to gate production deployments.

12. Summary & Checklist

Deployment Checklist

- ☐ Monaco CLI installed and `monaco version` returns output
- ☐ Platform token created with `slo:slos:read` / `slo:slos:write` (plus an API token with the "Configuration as Code" template scopes for Settings 2.0)
- ☐ `manifest.yaml` created with correct environment variable references
- ☐ Downloaded existing configuration with `monaco download`
- ☐ Explored directory structure and understood `config.yaml` format
- ☐ Created a new configuration (an `slo-v2` SLO with a DQL SLI) from scratch
- ☐ Validated with `monaco deploy --dry-run` (no errors)
- ☐ Deployed to target environment with `monaco deploy`
- ☐ Verified deployed config in Dynatrace UI
- ☐ Tested delete workflow with `delete.yaml`
- ☐ Configured multi-environment promotion with overrides

- [] Created GitHub Actions workflow for CI/CD

Key Takeaways

Concept	Summary
Manifest	Entry point defining projects and environments
Config Types	<code>slo-v2</code> , <code>segment</code> , <code>automation</code> , <code>document</code> , <code>bucket</code> , <code>settings</code> , <code>classic</code> — platform types need <code>auth.platformToken</code> or <code>auth.oAuth</code>
Download	Export existing tenant config to versioned files
Dry-Run	Always dry-run before deploying — structure and references only, no API calls
Deploy	Idempotent -- re-running updates in place
Delete	Only removes Monaco-managed configs
Overrides	<code>environmentOverrides</code> / <code>groupOverrides</code> for per-environment values and <code>skip</code>
CI/CD	Validate on PR, deploy on merge to main

References

- [Monaco Documentation](#)
- [Monaco GitHub Repository](#)
- [Monaco API Coverage](#)
- [Settings Schema Reference](#)
- [Monaco commands reference \(DT docs\)](#)
- [Monaco configuration YAML reference \(DT docs\)](#)
- [Settings 2.0 schemas that are removed in Latest Dynatrace \(DT docs\)](#)

Return to AUTOM-03: Monaco Configuration-as-Code for the lecture content, or continue to AUTOM-04: Terraform Provider.

Community Resources

Repository	Description
dynatrace-configuration-as-code	Official Monaco CLI (v2.29.1 at time of writing, 09/2026)
dynatrace-configuration-as-code-samples	9 starter templates in <code>basic-templates-monaco</code> , plus pipeline observability samples
easytrade	Real-world Monaco project structure with manifest.yaml
Dynatrace-Config-Manager	GUI tool for tenant-to-tenant config migration
monaco-self-paced-exercises	6 structured exercises (install, auto-tag, download, variables, delete/restore, linking)
monaco-demo	Working GitHub Actions workflow for Monaco deploy

Community Resources & Examples

After completing the hands-on steps above, these repositories provide deeper examples and exercises to reinforce what you've practiced:

Official Repositories

Repository	Description
dynatrace-configuration-as-code	Official Monaco CLI (v2.29.x at time of writing, 09/2026) — Go binary, Apache-2.0
dynatrace-configuration-as-code-samples	Official samples repo with 9 Monaco starter templates in <code>basic-templates-monaco</code>
easytrade	Demo microservices app with a working <code>monaco/</code> directory (manifest.yaml, detection rules, workflows)
Dynatrace-Config-Manager	GUI tool for tenant-to-tenant config migration; complements Monaco for brownfield scenarios

Training & Exercises (recommended next)

Repository	Description
monaco-self-paced-exercises	6 structured exercises: install, auto-tag, download, variables, delete/restore, linking configs
monaco-demo	Working GitHub Actions workflow for Monaco deploy with "crawl-walk-run" adoption methodology

Pipeline Observability Samples

Monaco configurations for ingesting CI/CD pipeline events via OpenPipeline:

Directory	CI/CD Platform
<code>github_pipeline_observability</code>	GitHub Actions
<code>gitlab_pipeline_observability</code>	GitLab CI
<code>azure_devops_observability</code>	Azure DevOps
<code>argocd_observability</code>	ArgoCD

Note: Monaco v1 (`dynatrace-oss/dynatrace-monitoring-as-code`) is no longer supported. Current Monaco releases have no conversion command: *"The last Monaco version to support the convert command is Monaco version 2.19.0."* Either run Monaco 2.19.0's `convert` per the [migrating to v2 guide \(DT docs\)](#), or re-run `monaco download` against the source tenant.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.